Übungsblatt 5

Ableitung und Optimierung

Skript: Abschnitte 4.4–4.6

Dieses Blatt übt den Stoff der fünften Vorlesung. Teil A sind kurze Rechenaufgaben mit konkreten Zahlen: „schließlich“ und die Landau-Symbole, die Ableitung, partielle Ableitungen mit Gradient und Hesse-Matrix, Jacobi-Matrix und Kettenregel, der Gradientenabstieg und Lagrange-Multiplikatoren. In Teil B kommen dieselben Themen in Lean: dieselben Begriffe, dieselben Funktionen, aber jetzt als Beweise, die der Compiler prüft.

Teil A: Auf Papier

Aufgabe A1 („Schließlich“ und Klein-o)

Zur Erinnerung: $f = o(g)$ längs eines Filters l heißt, dass zu *jedem* $c > 0$ schließlich $|f(x)| \le c\,|g(x)|$ gilt; bei $f = O(g)$ genügt *ein* c. Ist $g(x) \neq 0$ schließlich, so gilt $f = o(g)$ genau dann, wenn $f/g \to 0$, und $f = O(g)$ genau dann, wenn f/g schließlich beschränkt ist. (Bei $\mathcal{N}(0)$ rechnet man mit $x \neq 0$; die Funktionen hier verschwinden in 0, dort stimmt die Ungleichung von selbst.)

(a) Geben Sie an:

(i) das kleinste ganze N, so dass $x^2 \ge 100\,x$ für alle $x \ge N$ gilt;

(ii) das kleinste ganze N, so dass $3x + 7 < x^2$ für alle $x \ge N$ gilt;

(iii) das kleinste ganze N, so dass $1/x \le 1/1000$ für alle $x \ge N$ gilt;

(iv) ein $\delta > 0$, so dass $x^2 > 3$ für alle x mit $|x - 2| < \delta$ gilt.

(b) Entscheiden Sie mit dem Quotienten f/g, ob $f = o(g)$ und ob $f = O(g)$ gilt:

	f	g	längs	$f = o(g)$?	$f = O(g)$?
1	x^2	x	$\mathcal{N}(0)$		
2	x^2	x	atTop		
3	$\sin x$	x	$\mathcal{N}(0)$		
4	$\sin x$	x	atTop		
5	$x^2 + x$	x^2	atTop		
6	$x^2 + x$	x^2	$\mathcal{N}(0)$		

(c) Zu jedem c gehört ein eigenes δ bzw. N:

(i) $x^3 = o(x^2)$ längs $\mathcal{N}(0)$: Geben Sie zu $c = \frac{1}{10}$ und zu $c = \frac{1}{100}$ ein δ an, so dass $|x^3| \le c\,x^2$ für $|x| < \delta$.

(ii) $x^2 = o(x^3)$ längs atTop: Geben Sie zu $c = \frac{1}{10}$ und zu $c = \frac{1}{100}$ ein N an, so dass $x^2 \le c\,|x^3|$ für $x \ge N$.

(iii) x^3 ist längs atTop *nicht* $o(x^2)$: Für welche x gilt $|x^3| \le 1 \cdot x^2$, und warum ist das kein Endstück?

(d) Zeigen Sie $2x^2 + 7x^3 = o(x)$ längs $\mathcal{N}(0)$, erst mit dem Quotienten, dann so: $x^2 = o(x)$ und $x^3 = o(x)$ gelten längs $\mathcal{N}(0)$. Mit welchem c' prüft man beide Summanden, damit die Summe zu $c = \frac{1}{10}$ höchstens $c\,|x|$ wird?

Hinweis: Zu (a): Kleinstes N heißt, dass die Ungleichung bei $N-1$ schon nicht mehr für alle größeren x gilt. Zu (c, iii): $x^3 \le x^2$ heißt $x \le 1$.

Aufgabe A2 (Die Ableitung)

Nach der Vorlesung hat f in x die Ableitung f', wenn der Fehler der Tangente, $f(y) - f(x) - (y-x)\,f'$, klein gegen $y - x$ ist, also $o(y-x)$ für $y \to x$.

(a) Sei $f(x) = x^3 - 6x^2 + 9x$. Berechnen Sie f' und alle Stellen mit $f'(x) = 0$. Welche Stellen kommen nach Fermat für ein lokales Extremum in Frage, und welche sind ein Maximum, welche ein Minimum?

(b) Bestimmen Sie die Tangente an $f(y) = y^2$ in $x = 3$, und rechnen Sie den Fehler der Tangente aus. Wie groß ist er bei $y = 3{,}1$? Für welche y ist er höchstens $\frac{1}{100}\,|y-3|$?

(c) Zeigen Sie aus der Definition: $f(y) = y^2 + 3y$ hat in x die Ableitung $2x + 3$. Machen Sie die Probe bei $x = 1$, $y = \frac{6}{5}$.

(d) Sei $g(y) = |y|$. Berechnen Sie die Sekantensteigungen $\frac{g(y)-g(0)}{y-0}$ für $y = \pm 0{,}1$ und $y = \pm 0{,}01$. Hat g in 0 eine Ableitung? Hat g in 0 ein lokales Minimum? Widerspricht das dem Satz von Fermat?

Hinweis: In (c) lässt sich der Fehler als Quadrat schreiben, wie in (b).

Aufgabe A3 (Partielle Ableitungen, Gradient, Hesse-Matrix)

Sei $f(x, y) = x^2 y + 3y^2$.

(a) Berechnen Sie $\dfrac{\partial f}{\partial x}$ und $\dfrac{\partial f}{\partial y}$ und den Gradienten $\nabla f(1, 2)$.

(b) Berechnen Sie die vier zweiten partiellen Ableitungen und die Hesse-Matrix in $(1, 2)$. Was fällt bei den gemischten Ableitungen auf?

(c) Berechnen Sie $f(1,2) + \nabla f(1,2) \cdot h$ für $h = (\frac{1}{10}, -\frac{1}{10})$ und vergleichen Sie mit $f(1{,}1;\ 1{,}9)$. Wie groß ist die Richtungsableitung von f in $(1, 2)$ in Richtung $v = (1, -1)$?

(d) Sei $k(x, y) = \dfrac{xy}{x^2 + y^2}$ für $(x, y) \neq (0, 0)$ und $k(0, 0) = 0$. Berechnen Sie $k(t, 0)$, $k(0, t)$ und $k(t, t)$ für $t \neq 0$. Was folgt für die partiellen Ableitungen in $(0, 0)$, und ist k in $(0, 0)$ stetig? Ist k dort differenzierbar?

Hinweis: Bei einer partiellen Ableitung hält man die anderen Variablen fest und leitet in einer Variablen ab. Zu (c): $Df(x)\,h = \nabla f(x) \cdot h$.

Aufgabe A4 (Jacobi-Matrix und Kettenregel)

Seien $F\colon \mathbb{R}^2 \to \mathbb{R}^2$, $F(x, y) = (x^2 - y^2,\ 2xy)$ und $g\colon \mathbb{R}^2 \to \mathbb{R}$, $g(u, v) = u^2 + v^2$.

(a) Berechnen Sie die Jacobi-Matrix $J_F(x, y)$ und ihre Determinante. Was sind beide in $(1, 2)$?

(b) Berechnen Sie $F(1,2) + J_F(1,2)\,h$ für $h = (\frac{1}{10}, -\frac{1}{10})$ und vergleichen Sie mit $F(1{,}1;\ 1{,}9)$.

(c) Berechnen Sie $J_{g\circ F}(1,2)$ mit der Kettenregel, $J_g(F(1,2)) \cdot J_F(1,2)$, und noch einmal direkt, nachdem Sie $g(F(x,y))$ vereinfacht haben.

(d) Seien $P(r,\varphi) = (r\cos\varphi,\ r\sin\varphi)$ und g wie oben. Berechnen Sie $J_P(2,\frac{\pi}{2})$ mit seiner Determinante und $J_{g\circ P}(2,\frac{\pi}{2})$ mit der Kettenregel. Direkt: Was ist $g(P(r,\varphi))$?

Hinweis: Die Zeilen der Jacobi-Matrix sind die Gradienten der Komponenten. Zu (c): $(x^2-y^2)^2+(2xy)^2 = (x^2+y^2)^2$.

Aufgabe A5 (Gradientenabstieg)

Der Gradientenabstieg mit Schrittweite $t>0$ ist $x_{k+1} = x_k - t\,\nabla f(x_k)$.

(a) Sei $f(x) = (x-2)^2$, $t=\frac{1}{4}$ und $x_0 = 6$. Berechnen Sie x_1, x_2, x_3 und zeigen Sie $x_{k+1}-2 = \frac{1}{2}(x_k-2)$. Wie viele Schritte braucht man, bis $|x_n-2| \le \frac{1}{1000}$ ist?

(b) Für welche Schrittweiten t konvergiert das Verfahren in (a) für jeden Startwert? Was passiert bei $t=1$ und bei $t=\frac{6}{5}$ mit $x_0=6$?

(c) Sei nun $f(x,y) = \frac{1}{2}(x^2+4y^2)$ mit Minimum in $(0,0)$. Berechnen Sie ∇f, dann zwei Schritte mit $t=\frac{1}{5}$ von $(x_0,y_0)=(4,1)$ aus, und geben Sie (x_n,y_n) für alle n an.

(d) Für welche t konvergiert das Verfahren in (c) von jedem Startwert aus gegen $(0,0)$? Was passiert bei $t=\frac{1}{2}$?

(e) Sei $\rho(t) = \max\{\,|1-t|,\ |1-4t|\,\}$; dann ist $\|(x_n,y_n)\| \le \rho(t)^n\,\|(x_0,y_0)\|$. Für welches t ist $\rho(t)$ am kleinsten? Wie viele Schritte garantiert die Schranke bis $\|(x_n,y_n)\| \le \frac{1}{1000}\,\|(x_0,y_0)\|$ mit diesem t, wie viele mit $t=\frac{1}{5}$?

Hinweis: In jedem Schritt wird der Fehler (bzw. jede Koordinate) mit einem festen Faktor multipliziert. Zu (e): $|1-t|$ und $|1-4t|$ über t skizzieren.

Aufgabe A6 (Lagrange)

(a) Minimieren Sie $\varphi(x,y) = x^2+y^2$ unter der Nebenbedingung $f(x,y) = x+2y = 5$: Prüfen Sie $\nabla f \neq 0$, stellen Sie $\nabla\varphi = \lambda\,\nabla f$ auf und bestimmen Sie λ und den Kandidaten p. Welchen Wert hat φ dort?

(b) Kontrollieren Sie das ohne Lagrange: Setzen Sie $x = 5-2y$ in φ ein.

(c) Bestimmen Sie die Lagrange-Kandidaten für $\varphi(x,y) = x+y$ unter $x^2+y^2=2$ und entscheiden Sie, wo das Maximum und wo das Minimum liegt.

Teil B: In Lean

Öffnen Sie die Datei `lean/Blatt05.lean` im Editor (VS Code mit der Lean-4-Erweiterung, in diesem Repository mit Mathlib), und ersetzen Sie jedes `sorry` durch einen Beweis; der Infoview zeigt Ziel und Kontext. Die Datei ist fertig, wenn Lean keine Warnung „declaration uses ‚sorry'" mehr meldet. Die Begriffe `KleinO` und `HatPartielleAbleitung` sind die der Vorlesung (`code/Vorlesung05.lean`), und ebenso die Beweise, auf die die Kommentare der Datei mit Namen verweisen. Beim Suchen helfen `exact?` und `apply?`.

Aufgabe B1 („Schließlich": zu A1 (a))

$\forall^{\mathrm{f}}$ `x in F, P x` heißt „$P(x)$ gilt schließlich längs F": Die Menge $\{x \mid P(x)\}$ liegt im Filter F. Längs `atTop` ist eine Menge groß, wenn sie ein Endstück $[N, \infty)$ enthält, längs $\mathcal{N}$ `a`, wenn sie eine Kugel um a enthält.

(a) Schließlich längs atTop ist $3x + 7 < x^2$ (A1 (a), ii).

(b) Schließlich längs $\mathcal{N}(2)$ ist $3 < x^2$ (A1 (a), iv).

(c) Gilt P schließlich und Q schließlich, und folgt R aus P und Q, so gilt R schließlich: die Filteraxiome (ii) und (iii) in einem Satz.

Hinweis: `eventually_ge_atTop N` ist das Endstück ab N als große Menge, `Metric.ball_mem_nhds a hε` die Kugel vom Radius ε um a. `filter_upwards [U, V] with x hx hy` macht aus dem Ziel „schließlich gilt …" eine Aussage über einen Punkt x, an dem die großen Mengen U und V beide gelten – der Schnitt aus Axiom (iii). Ungleichungen mit Produkten erledigt `nlinarith`.

Lean lean/Blatt05.lean

```
-- ‘∀ᶠ x in F, P x‘ heisst: Die Menge {x | P x} ist gross, sie liegt im Filter F.
-- Laengs ‘atTop‘ ist gross, was ein Endstueck [N, ∞) enthaelt, laengs ‘𝒩 a‘,
-- was eine Kugel um a enthaelt.

/-- Aufgabe B1 (a) (A1 (a), ii): Schliesslich laengs ‘atTop‘ ist
    ‘3x + 7 < x²‘. -/
theorem schliesslich_quadrat : ∀ᶠ x : ℝ in atTop, 3 * x + 7 < x ^ 2 := by
  -- Nuetzlich: eventually_ge_atTop (das Endstueck [N, ∞)), filter_upwards,
    nlinarith
  sorry

/-- Aufgabe B1 (b) (A1 (a), iv): Schliesslich laengs ‘𝒩 2‘ ist
    ‘3 < x²‘. -/
theorem schliesslich_nahe_zwei : ∀ᶠ x : ℝ in 𝒩 2, 3 < x ^ 2 := by
  -- Nuetzlich: Metric.ball_mem_nhds (2 : ℝ) (die Kugel vom Radius 1/4),
  -- Metric.mem_ball, Real.dist_eq, abs_lt, nlinarith
  sorry

/-- Aufgabe B1 (c): Gilt ‘P‘ schliesslich und ‘Q‘ schliesslich, und folgt ‘R‘
    aus ‘P‘ und ‘Q‘, so gilt ‘R‘ schliesslich -- die Filteraxiome (ii) und
    (iii) in einem Satz. -/
theorem schliesslich_folgt {α : Type*} {F : Filter α} {P Q R : α → Prop}
    (hP : ∀ᶠ x in F, P x) (hQ : ∀ᶠ x in F, Q x) (h : ∀ x, P x → Q x → R x) :
    ∀ᶠ x in F, R x := by
  -- Nuetzlich: filter_upwards [hP, hQ] with x hp hq
  sorry
```

Aufgabe B2 (Klein-o: zu A1 (c) und (d))

Die Definition `KleinO f g l` steht am Anfang des Lean-Kastens: zu jedem $c > 0$ schließlich $|f(x)| \le c\,|g(x)|$.

(a) $x^3 = o(x^2)$ längs $\mathcal{N}(0)$.

(b) $x^2 = o(x^3)$ längs atTop.

(c) x^3 ist längs atTop nicht $o(x^2)$.

(d) Die Linearkombination aus A1 (d): Sind $f_1 = o(g)$ und $f_2 = o(g)$ längs eines beliebigen Filters l und $a, b \in \mathbb{R}$, so ist $af_1 + bf_2 = o(g)$.

Hinweis: Die Beweise gehen wie die zu x^2 und x in der Vorlesung. Große Mengen liefern `Metric.ball_mem_nhds 0 hc` (die Kugel vom Radius c um 0) und `eventually_ge_atTop N` (das Endstück ab N). In (c) braucht man einen Punkt, an dem zwei Aussagen zugleich gelten; aus $\mathtt{h}_1$ und $\mathtt{h}_2$ macht `(h₁.and h₂).exists` ihn. In (d) bildet `filter_upwards` den Schnitt der beiden Mengen zu $c' = c/(|a| + |b| + 1)$. Ungleichungen mit Produkten erledigt `nlinarith`, wenn man ihm `sq_nonneg x` mitgibt.

Lean `lean/Blatt05.lean`

```
/-- Klein-o laengs eines Filters, wie in der Vorlesung: Zu jedem ‘c > 0‘ gilt
    schliesslich ‘|f x| ≤ c·|g x|‘. -/
def KleinO {α : Type*} (f g : α → ℝ) (l : Filter α) : Prop :=
  ∀ c > 0, ∀ᶠ x in l, |f x| ≤ c * |g x|

/-- Aufgabe B2 (a) (A1 (c), i): Bei ‘0‘ ist ‘x³‘ klein gegen ‘x²‘. -/
theorem kubik_klein_o_null :
    KleinO (fun x : ℝ => x ^ 3) (fun x => x ^ 2) (𝓝 0) := by
  -- Nuetzlich: wie quadrat_klein_o in der Vorlesung: intro c hc,
  -- filter_upwards [Metric.ball_mem_nhds (0 : ℝ) hc] with x hx, Metric.mem_ball,
  -- Real.dist_eq, sub_zero, abs_pow, positivity, nlinarith
  sorry

/-- Aufgabe B2 (b) (A1 (c), ii): Bei ‘+∞‘ ist es umgekehrt: ‘x²‘ ist
    klein gegen ‘x³‘. -/
theorem quadrat_klein_o_kubik :
    KleinO (fun x : ℝ => x ^ 2) (fun x => x ^ 3) atTop := by
  -- Nuetzlich: wie linear_klein_o_unendlich in der Vorlesung:
  -- filter_upwards [eventually_ge_atTop (1 / c)] with x hx, div_le_iff₀,
  -- positivity, abs_of_nonneg, sq_nonneg, pow_nonneg, nlinarith
  sorry

/-- Aufgabe B2 (c) (A1 (c), iii): Bei ‘+∞‘ ist ‘x³‘ nicht klein gegen
    ‘x²‘. -/
theorem kubik_nicht_klein_o :
    ¬ KleinO (fun x : ℝ => x ^ 3) (fun x => x ^ 2) atTop := by
  -- Nuetzlich: wie quadrat_nicht_klein_o_unendlich in der Vorlesung: c = 1
  -- (one_pos); Filter.Eventually.and und .exists liefern einen Punkt x, an
  -- dem die Ungleichung und x ≥ 2 (eventually_ge_atTop) zugleich gelten
  sorry

/-- Aufgabe B2 (d) (A1 (d)): Linearkombinationen von ‘o(g)‘ sind ‘o(g)‘ -- mit
    dem Schnitt zweier grosser Mengen. -/
theorem klein_o_linear {α : Type*} {l : Filter α} {f₁ f₂ g : α → ℝ} (a b : ℝ)
    (h₁ : KleinO f₁ g l) (h₂ : KleinO f₂ g l) :
    KleinO (fun x => a * f₁ x + b * f₂ x) g l := by
  -- Nuetzlich: wie klein_o_add in der Vorlesung, mit c / (|a| + |b| + 1)
```

```
  -- statt c / 2; positivity, div_pos, filter_upwards [h₁ _ _, h₂ _ _],
  -- abs_add_le, abs_mul, gcongr, div_le_one, calc
  sorry
```

Aufgabe B3 (Die Ableitung: zu A2 (c) und (d))

Vorgegeben sind `quadrat_klein_o` ($(y-x)^2$ ist klein gegen $y-x$) und `ableitung_klein_o` (die Definition der Ableitung: Der Fehler der Tangente ist klein gegen $y-x$), beide wie in der Vorlesung.

(a) y^2+3y hat in x die Ableitung $2x+3$ – aus der Definition, wie y^2 in der Vorlesung.

(b) Der Betrag ist in 0 nicht klein gegen y, d. h. $|y| = o(y)$ längs $\mathcal{N}(0)$ gilt nicht. Das ist die Rechnung aus A2 (d) mit der Steigung 0.

(c) Der Betrag hat in 0 keine Ableitung.

Hinweis: In (a) ist der Fehler der Tangente $(y-x)^2$; `funext y` und `ring` zeigen das. In (b): `Metric.eventually_nhds_iff` macht aus „schließlich längs $\mathcal{N}(0)$“ eine Kugel vom Radius δ; der Punkt $\delta/2$ liegt darin. In (c) hat der Betrag in 0 ein lokales Minimum (`IsLocalMin`), nach Fermat – in Mathlib `IsLocalMin.hasDerivAt_eq_zero`, in der Vorlesung bewiesen – wäre die Ableitung also 0.

Lean `lean/Blatt05.lean`

```
-- Vorgegeben, wie in der Vorlesung: Bei ‘x‘ ist ‘(y - x)^2‘ klein gegen ‘y - x‘,
-- und ‘f‘ hat in ‘x‘ die Ableitung ‘f’‘, wenn der Fehler der Tangente klein
-- gegen ‘y - x‘ ist.

/-- Bei ‘x‘ ist ‘(y - x)^2‘ klein gegen ‘y - x‘: Zu jedem ‘c‘ reicht die Kugel
    vom Radius ‘c‘ um ‘x‘. -/
theorem quadrat_klein_o (x : ℝ) :
    KleinO (fun y : ℝ => (y - x) ^ 2) (fun y => y - x) (𝓝 x) := by
  intro c hc
  filter_upwards [Metric.ball_mem_nhds x hc] with y hy
  rw [Metric.mem_ball, Real.dist_eq] at hy
  rw [abs_pow, sq]
  exact mul_le_mul_of_nonneg_right hy.le (abs_nonneg _)

/-- Die Ableitung, wie auf der Folie: Der Fehler der Tangente ist klein gegen
    ‘y - x‘ laengs ‘𝓝 x‘. -/
theorem ableitung_klein_o (f : ℝ → ℝ) (f' x : ℝ) :
    HasDerivAt f f' x ↔
      KleinO (fun y => f y - f x - (y - x) * f') (fun y => y - x) (𝓝 x) := by
  rw [hasDerivAt_iff_isLittleO, Asymptotics.isLittleO_iff]
  simp only [KleinO, Real.norm_eq_abs, smul_eq_mul, gt_iff_lt]

/-- Aufgabe B3 (a) (A2 (c)): ‘y^2 + 3y‘ hat in ‘x‘ die Ableitung ‘2x + 3‘ --
    aus der Definition, wie ‘y^2‘ in der Vorlesung. -/
theorem ableitung_quadrat_plus (x : ℝ) :
    HasDerivAt (fun y : ℝ => y ^ 2 + 3 * y) (2 * x + 3) x := by
  -- Nuetzlich: rw [ableitung_klein_o]; der Fehler der Tangente ist (y - x)^2
  -- (have fehler : (fun y => ...) = fun y => (y - x) ^ 2, ext y, ring);
  -- dann quadrat_klein_o
  sorry

/-- Aufgabe B3 (b) (A2 (d)): Der Betrag ist in ‘0‘ nicht klein gegen ‘y‘:
    Mit ‘c = 1/2‘ muesste schliesslich ‘|y| ≤ (1/2)·|y|‘ gelten. -/
theorem betrag_nicht_klein_o :
    ¬ KleinO (fun y : ℝ => |y|) (fun y => y) (𝓝 0) := by
```

```
  -- Nuetzlich: intro h; h (1 / 2) (by norm_num); Metric.eventually_nhds_iff
  -- liefert ein δ > 0; der Punkt δ / 2 liegt in der Kugel (Real.dist_eq,
  -- abs_of_pos, half_pos) und verletzt die Ungleichung; abs_abs, linarith
  sorry

/-- Aufgabe B3 (c) (A2 (d)): Der Betrag hat in ‘0‘ keine Ableitung. -/
theorem betrag_keine_ableitung :
    ¬ ∃ f' : ℝ, HasDerivAt (fun y : ℝ => |y|) f' 0 := by
  -- Nuetzlich: In 0 liegt ein lokales Minimum (IsLocalMin, Eventually.of_forall),
  -- also ist f' = 0 nach Fermat: IsLocalMin.hasDerivAt_eq_zero. Dann
  -- ableitung_klein_o und (b); simpa
  sorry
```

Aufgabe B4 (Partielle Ableitungen: zu A3 (a) und (d))

Punkte der Ebene sind in diesem Teil Vektoren `v : Fin 2 → ℝ` mit den Koordinaten `v 0` und `v 1`. `HatPartielleAbleitung f x j a` heißt: Die Schnittkurve $t \mapsto f(x + t\,e_j)$ hat in 0 die Ableitung a. Das Beispiel x^2y nach x aus der Vorlesung steht als `example` in der Datei.

(a) $f(x, y) = x^2y + 3y^2$ hat in v nach y die partielle Ableitung $x^2 + 6y$.

(b) Das Gegenbeispiel $k(x, y) = \frac{xy}{x^2+y^2}$ aus A3 (d) ist in Lean `gegenbeispiel`, mit $k(0, 0) = 0$, weil in Lean $a/0 = 0$ ist. Seine partielle Ableitung nach y in 0 ist 0.

(c) Auf der Diagonalen ist $k(t, t) = \frac{1}{2}$ für $t \neq 0$.

Hinweis: (a) geht wie das Beispiel in der Datei: `unfold HatPartielleAbleitung`, dann die Schnittkurve ausrechnen (`funext t`, `simp`), dann die Regeln `hasDerivAt_id`, `.const_add`, `.pow`, `.const_mul` und `.add`, und zuletzt `convert … using 1`. (b) geht wie die partielle Ableitung nach x in der Vorlesung: Die Schnittkurve ist die Nullfunktion, `hasDerivAt_const`. (c) `field_simp` und `ring`.

Lean lean/Blatt05.lean

```
/-- Partielle Ableitung, wie in der Vorlesung: ‘f‘ hat in ‘x‘ nach der
    ‘j‘-ten Koordinate die partielle Ableitung ‘a‘, wenn die Schnittkurve
    ‘t ↦ f(x + t·e_j)‘ in ‘0‘ die Ableitung ‘a‘ hat. Punkte der Ebene sind
    Vektoren ‘v : Fin 2 → ℝ‘ mit den Koordinaten ‘v 0‘ und ‘v 1‘. -/
def HatPartielleAbleitung {n : ℕ} {F : Type*} [NormedAddCommGroup F] [NormedSpace ℝ
    F]
    (f : (Fin n → ℝ) → F) (x : Fin n → ℝ) (j : Fin n) (a : F) : Prop :=
  HasDerivAt (fun t : ℝ => f (x + t • (Pi.single j 1 : Fin n → ℝ))) a 0

/-- Vorgegeben: das Beispiel der Vorlesung, ‘f(x, y) = x²y‘ nach ‘x‘. -/
example (v : Fin 2 → ℝ) :
    HatPartielleAbleitung (fun w : Fin 2 → ℝ => w 0 ^ 2 * w 1) v 0 (2 * v 0 * v 1)
    := by
  unfold HatPartielleAbleitung
  have e : (fun t : ℝ => (v + t • (Pi.single 0 1 : Fin 2 → ℝ)) 0 ^ 2 *
      (v + t • (Pi.single 0 1 : Fin 2 → ℝ)) 1) = fun t => (v 0 + t) ^ 2 * v 1 := by
    funext t
    simp
  rw [e]
  have h := (((hasDerivAt_id (0 : ℝ)).const_add (v 0)).pow 2).mul_const (v 1)
  convert h using 1
  simp

/-- Aufgabe B4 (a) (A3 (a)): ‘f(x, y) = x²y + 3y²‘ hat in ‘v‘ die partielle
    Ableitung ‘x² + 6y‘ nach ‘y‘. -/
theorem partiell_nach_y (v : Fin 2 → ℝ) :
```

```
    HatPartielleAbleitung (fun w : Fin 2 → ℝ => w 0 ^ 2 * w 1 + 3 * w 1 ^ 2) v 1
      (v 0 ^ 2 + 6 * v 1) := by
  -- Nuetzlich: wie das Beispiel x²y oben: unfold HatPartielleAbleitung; die
  -- Schnittkurve ausrechnen (funext t, simp); hasDerivAt_id, .const_add, .pow,
  -- .const_mul, .add; convert ... using 1; simp; ring
  sorry

/-- In Lean ist ‘a / 0 = 0‘, also ‘f(0, 0) = 0‘ von selbst. -/
noncomputable def gegenbeispiel (w : Fin 2 → ℝ) : ℝ :=
  w 0 * w 1 / (w 0 ^ 2 + w 1 ^ 2)

/-- Aufgabe B4 (b) (A3 (d)): Auf der ‘y‘-Achse ist ‘f‘ null; die partielle
    Ableitung nach ‘y‘ in ‘0‘ ist ‘0‘. -/
theorem gegenbeispiel_partiell_y : HatPartielleAbleitung gegenbeispiel 0 1 0 := by
  -- Nuetzlich: wie die partielle Ableitung nach x in der Vorlesung: Die
  -- Schnittkurve ist die Nullfunktion (funext t, simp [gegenbeispiel]);
  -- hasDerivAt_const
  sorry

/-- Aufgabe B4 (c) (A3 (d)): Auf der Diagonalen ist ‘f = 1/2‘, ausser in ‘0‘. -/
theorem gegenbeispiel_diagonale {t : ℝ} (ht : t ≠ 0) :
    gegenbeispiel ![t, t] = 1 / 2 := by
  -- Nuetzlich: simp only [gegenbeispiel, Matrix.cons_val_zero,
  -- Matrix.cons_val_one],
  -- field_simp, ring
  sorry
```

Aufgabe B5 (Jacobi-Matrix und Kettenregel: zu A4 (a) und (c))

Punkte der Ebene sind Vektoren `v : Fin 2 → ℝ` mit den Koordinaten `v 0` und `v 1`; `![a, b]` ist der Vektor (a, b), `!![a, b; c, d]` die Matrix mit den Zeilen (a, b) und (c, d). Vorgegeben sind wie in der Vorlesung `matrixAbb`, das aus einer Matrix die lineare Abbildung $h \mapsto Mh$ macht, das Lemma `matrixAbb_comp` (Hintereinanderausführen ist Matrizenmultiplizieren) und `jacobi_quadratsumme`, die Jacobi-Matrix $(2u,\ 2v)$ von $g(u, v) = u^2 + v^2$.

(a) $F(x, y) = (x^2 - y^2,\ 2xy)$ hat die Jacobi-Matrix aus A4 (a).

(b) Kettenregel: $g \circ F$ hat die Jacobi-Matrix $J_g(F(v)) \cdot J_F(v)$.

(c) Das Produkt ausgerechnet ist $4\,(x^2 + y^2)\,(x,\ y)$ – der Gradient aus A4 (c).

Hinweis: (a) geht wie `jacobi_xy` in der Vorlesung: `hasFDerivAt_pi'` zerlegt die Aussage in eine je Zeile, `fin_cases i` unterscheidet die Zeilen, `hasFDerivAt_apply 0 v` ist die Ableitung der Koordinate `w 0`; dazu `.pow`, `.sub`, `.const_mul` und `.mul`. (b) wie `kettenregel_polar` in der Vorlesung. (c) `ext i j` und `fin_cases` gehen die Einträge durch; `simp` mit `Matrix.mul_apply` und `Fin.sum_univ_two` rechnet sie aus.

Lean lean/Blatt05.lean

```
-- Vorgegeben, wie in der Vorlesung: eine Matrix als lineare Abbildung, das
-- Hintereinanderausfuehren als Matrixprodukt und die Jacobi-Matrix von
-- u² + v². Punkte sind Vektoren ‘v : Fin 2 → ℝ‘ mit ‘v 0 = x‘, ‘v 1 = y‘.

/-- Eine m×n-Matrix als lineare Abbildung ℝⁿ → ℝᵐ, h ↦ M h. -/
noncomputable def matrixAbb {m n : ℕ} (M : Matrix (Fin m) (Fin n) ℝ) :
    (Fin n → ℝ) →L[ℝ] (Fin m → ℝ) :=
  LinearMap.toContinuousLinearMap (Matrix.toLin' M)

/-- Angewandt auf h ist sie Matrix mal Vektor. -/
```

```
@[simp] lemma matrixAbb_apply {m n : ℕ} (M : Matrix (Fin m) (Fin n) ℝ)
    (h : Fin n → ℝ) : matrixAbb M h = M.mulVec h := rfl

/-- Hintereinander ausfuehren heisst Matrizen multiplizieren. -/
lemma matrixAbb_comp {m n k : ℕ} (A : Matrix (Fin m) (Fin n) ℝ)
    (B : Matrix (Fin n) (Fin k) ℝ) :
    (matrixAbb A).comp (matrixAbb B) = matrixAbb (A * B) := by
  ext h : 1
  simp [Matrix.mulVec_mulVec]

/-- g(u, v) = u² + v², als Abbildung nach ℝ¹: Jacobi-Matrix (2u, 2v). -/
theorem jacobi_quadratsumme (u : Fin 2 → ℝ) :
    HasFDerivAt (fun w : Fin 2 → ℝ => ![w 0 ^ 2 + w 1 ^ 2])
      (matrixAbb ![2 * u 0, 2 * u 1]) u := by
  rw [hasFDerivAt_pi']
  intro i
  fin_cases i
  have h := ((hasFDerivAt_apply (𝕜 := ℝ) 0 u).pow 2).add
    ((hasFDerivAt_apply (𝕜 := ℝ) 1 u).pow 2)
  convert h using 1
  ext k
  simp [dotProduct, Fin.sum_univ_two]

/-- Aufgabe B5 (a) (A4 (a)): ‘F(x, y) = (x² - y², 2xy)‘ hat die Jacobi-Matrix
    ‘[[2x, -2y], [2y, 2x]]‘. -/
theorem jacobi_quadrat (v : Fin 2 → ℝ) :
    HasFDerivAt (fun w : Fin 2 → ℝ => ![w 0 ^ 2 - w 1 ^ 2, 2 * w 0 * w 1])
      (matrixAbb ![2 * v 0, -(2 * v 1); 2 * v 1, 2 * v 0]) v := by
  -- Nuetzlich: wie jacobi_xy in der Vorlesung: rw [hasFDerivAt_pi'], intro i,
  -- fin_cases i, hasFDerivAt_apply (𝕜 := ℝ) 0 v, HasFDerivAt.pow, .sub,
  -- .const_mul, .mul, convert ... using 1, ext k,
  -- simp [dotProduct, Fin.sum_univ_two], ring
  sorry

/-- Aufgabe B5 (b) (A4 (c), Kettenregel): ‘g ∘ F‘ hat die Jacobi-Matrix
    ‘J_g(F(v)) · J_F(v)‘. -/
theorem kette_quadrat (v : Fin 2 → ℝ) :
    HasFDerivAt (fun w : Fin 2 → ℝ => ![(w 0 ^ 2 - w 1 ^ 2) ^ 2 + (2 * w 0 * w 1)
    ^ 2])
      (matrixAbb (![2 * (v 0 ^ 2 - v 1 ^ 2), 2 * (2 * v 0 * v 1)] *
        ![2 * v 0, -(2 * v 1); 2 * v 1, 2 * v 0])) v := by
  -- Nuetzlich: wie kettenregel_polar in der Vorlesung: HasFDerivAt.comp mit
  -- jacobi_quadratsumme an der Stelle ![v 0 ^ 2 - v 1 ^ 2, 2 * v 0 * v 1] und
  -- jacobi_quadrat v; matrixAbb_comp; simpa
  sorry

/-- Aufgabe B5 (c) (A4 (c), Rechnung): Das Produkt der Jacobi-Matrizen ist
    ‘4(x² + y²)·(x, y)‘ -- der Gradient von ‘(x² + y²)²‘. -/
theorem produkt_quadrat (x y : ℝ) :
    ![2 * (x ^ 2 - y ^ 2), 2 * (2 * x * y)] * ![2 * x, -(2 * y); 2 * y, 2 * x] =
      ![4 * x * (x ^ 2 + y ^ 2), 4 * y * (x ^ 2 + y ^ 2)] := by
  -- Nuetzlich: ext i j, fin_cases i, fin_cases j,
  -- simp [Matrix.mul_apply, Fin.sum_univ_two], ring
  sorry
```

Aufgabe B6 (Gradientenabstieg: zu A5 (c) und (d))

Punkte der Ebene sind in diesem Teil Paare `p` mit den Koordinaten `p.1` und `p.2`. Auf Paaren rechnet Mathlib mit der Maximumsnorm statt der euklidischen Norm; für die Konvergenz macht das keinen Unterschied – beide Male heißt sie: Konvergenz beider Koordinaten. Vorgegeben sind der Gradient von $f(x,y) = \frac{1}{2}(ax^2 + by^2)$ als `gradF` (A5 hat $a = 1$, $b = 4$; dass er der Gradient von f ist, beweisen wir in Lean nicht), ein Schritt `gdSchritt2` und die Iteration `gdIter2`, gebaut wie `gdSchritt` und `gdIter` in der Vorlesung. Vorgegeben ist auch das Lemma `gd2_schritt`: Ein Schritt multipliziert die Koordinaten mit $1 - ta$ bzw. $1 - tb$.

(a) Die geschlossene Form aus A5 (c): $x_n = (1-ta)^n\, x_0$ und $y_n = (1-tb)^n\, y_0$.

(b) $|1 - ta| < 1$ und $|1 - tb| < 1$ gelten genau dann, wenn $0 < t < 2/b$ – für $0 < a \le b$.

(c) Für $0 < t < 2/b$ konvergiert das Verfahren von jedem Startwert aus gegen $(0,0)$ – die Richtung „⇐“ von A5 (d).

Hinweis: (a) geht wie `gd_q_fehler` aus der Vorlesung; nach `ext` ist für jede Koordinate eine Gleichung zwischen reellen Zahlen übrig. (b) geht wie `schrittweite` aus der Vorlesung. In (c) liefert (b) die beiden Beträge < 1; dann weiter wie in `gd_q_tendsto`. `Tendsto.prodMk_nhds` setzt die Grenzwerte der beiden Koordinaten zu einem Grenzwert in der Ebene zusammen.

Lean `lean/Blatt05.lean`

```
/-- Der Gradient von `f(x, y) = (a x^2 + b y^2)/2` -- fuer `a = 1`, `b = 4` die
    Funktion aus A5. Punkte der Ebene sind Paare `p` mit den Koordinaten `p.1`
    und `p.2`. -/
def gradF (a b : ℝ) (p : ℝ × ℝ) : ℝ × ℝ := (a * p.1, b * p.2)

/-- Ein Schritt des Gradientenabstiegs, `p - t ∇f(p)` (wie gdSchritt). -/
def gdSchritt2 (a b t : ℝ) (p : ℝ × ℝ) : ℝ × ℝ := p - t • gradF a b p

/-- Die Iteration `p₀, p₁, p₂, ...` (wie gdIter in der Vorlesung). -/
def gdIter2 (a b t : ℝ) (p₀ : ℝ × ℝ) : ℕ → ℝ × ℝ
  | 0 => p₀
  | n + 1 => gdSchritt2 a b t (gdIter2 a b t p₀ n)

/-- Vorgegeben: Ein Schritt multipliziert die Koordinaten mit `1 - t a` bzw.
    `1 - t b`. `ext` zerlegt eine Gleichung zwischen Paaren in zwei
    Gleichungen, eine fuer jede Koordinate. -/
theorem gd2_schritt (a b t : ℝ) (p : ℝ × ℝ) :
    gdSchritt2 a b t p = ((1 - t * a) * p.1, (1 - t * b) * p.2) := by
  ext
  · simp [gdSchritt2, gradF]
    ring
  · simp [gdSchritt2, gradF]
    ring

/-- Aufgabe B6 (a) (A5 (c)): die geschlossene Form. -/
theorem gd2_formel (a b t : ℝ) (p₀ : ℝ × ℝ) (n : ℕ) :
    gdIter2 a b t p₀ n = ((1 - t * a) ^ n * p₀.1, (1 - t * b) ^ n * p₀.2) := by
  -- Nuetzlich: wie gd_q_fehler in der Vorlesung, mit gd2_schritt; fuer
  -- Gleichungen zwischen Paaren hilft ext (siehe gd2_schritt)
  sorry

/-- Aufgabe B6 (b) (A5 (d), Rechnung): Beide Faktoren haben genau dann Betrag
    `< 1`, wenn `0 < t < 2/b`. -/
theorem schrittweite2 (a b t : ℝ) (ha : 0 < a) (hab : a ≤ b) :
    (|1 - t * a| < 1 ∧ |1 - t * b| < 1) ↔ (0 < t ∧ t < 2 / b) := by
```

```
  -- Nuetzlich: wie schrittweite in der Vorlesung: abs_lt, lt_div_iff₀
  -- (dafuer erst 0 < b zeigen), constructor, rintro, nlinarith
  sorry

/-- Aufgabe B6 (c) (A5 (d), eine Richtung): Fuer `0 < t < 2/b` konvergiert das
    Verfahren von jedem Startwert aus gegen das Minimum `(0, 0)`. -/
theorem gd2_konvergenz (a b t : ℝ) (ha : 0 < a) (hab : a ≤ b) (ht : 0 < t)
    (ht' : t < 2 / b) (p₀ : ℝ × ℝ) :
    Tendsto (gdIter2 a b t p₀) atTop (𝓝 (0, 0)) := by
  -- Nuetzlich: schrittweite2; aus gd_q_tendsto in der Vorlesung
  -- tendsto_pow_atTop_nhds_zero_of_norm_lt_one, Real.norm_eq_abs und
  -- Tendsto.mul_const; Tendsto.prodMk_nhds; funext (gd2_formel a b t p₀)
  -- ist die Formel als Gleichung zwischen Folgen
  sorry
```

Aufgabe B7 (Lagrange: zu A6 (a) und (b))

Lagrange liefert den Kandidaten; dass er wirklich das Optimum ist, bestätigt eine elementare Ungleichung, wie bei $xy \le \frac{1}{4}$ unter $x + y = 1$ in der Vorlesung.

(a) Auf der Geraden $x + 2y = 5$ ist $x^2 + y^2 \ge 5$.

(b) Der Wert 5 wird nur im Lagrange-Punkt $(1, 2)$ angenommen.

Hinweis: Es ist $5\,(x^2 + y^2) = (x + 2y)^2 + (2x - y)^2$. Geben Sie `nlinarith` den Hinweis `sq_nonneg (2 * x - y)`. In (b) ist dann $(2x - y)^2 = 0$, und aus `pow_eq_zero_iff` folgt $2x - y = 0$.

Lean lean/Blatt05.lean

```
/-- Aufgabe B7 (a) (A6 (a)): Auf der Geraden `x + 2y = 5` ist `x² + y² ≥ 5`;
    der Lagrange-Kandidat `(1, 2)` liefert den Wert `5`. -/
theorem abstand_gerade (x y : ℝ) (h : x + 2 * y = 5) : 5 ≤ x ^ 2 + y ^ 2 := by
  -- Nuetzlich: wie xy_le_quarter in der Vorlesung: nlinarith mit dem Hinweis
  -- sq_nonneg (2 * x - y); denn 5·(x² + y²) = (x + 2y)² + (2x - y)²
  sorry

/-- Aufgabe B7 (b) (A6 (a)): Der Wert `5` wird nur in `(1, 2)` angenommen. -/
theorem abstand_gerade_eindeutig (x y : ℝ) (h : x + 2 * y = 5)
    (hmin : x ^ 2 + y ^ 2 = 5) : x = 1 ∧ y = 2 := by
  -- Nuetzlich: Aus der Gleichung 5·(x² + y²) = (x + 2y)² + (2x - y)² folgt
  -- (2x - y)² = 0 (nlinarith), also 2x - y = 0 (pow_eq_zero_iff); dann linarith
  sorry
```